OMNISCIA

August 1st, 2026

Smart Contract Audit Report

UltraYield

Vault Contracts

WEB omniscia.io

EMAIL info@omniscia.io

ONLINE REPORT https://omniscia.io/reports/ultrayield-vault-contracts-6a50edfdfba55a00167aab50

ABOUT OMNISCIA

Omniscia.io is one of the fastest growing and most trusted blockchain security firms and has rapidly become a true market leader. To date, our team has collectively secured over 370+ clients, detecting 1,500+ high-severity issues in widely adopted smart contracts.

Founded in France at the start of 2020, and with a track record spanning back to 2017, our team has been at the forefront of auditing smart contracts, providing expert analysis and identifying potential vulnerabilities to ensure the highest level of security of popular smart contracts, as well as complex and sophisticated decentralized protocols.

Our clients, ecosystem partners, and backers include leading ecosystem players such as L'Oréal, Polygon, AvaLabs, Gnosis, Morpho, Vesta, Gravita, Olympus DAO, Fetch.ai, and LimitBreak, among others.

To keep up to date with all the latest news and announcements follow us on twitter @omniscia_sec.

WEB	omniscia.io
EMAIL	info@omniscia.io
ONLINE REPORT	https://omniscia.io/reports/ultrayield-vault-contracts-6a50edfdfba55a00167aab50

SECURITY AUDIT REPORT

Vault Contracts Security Audit

A specialized vault implementation for the UltraYield contract system.

FINDINGS	ADDRESSED	HIGHEST SEVERITY	REVISIONS
33	26	Medium	4

Audit Report Revisions

INITIAL REVIEW July 6th, 2026

COMMIT `46cb108cf7` ❖ `2524e96319`

33 identified

REMEDIATION 01 July 15th, 2026

COMMIT `d0f8fd98b7` ❖ `14e5a922ce`

23 alleviated · 2 partially alleviated · 6 acknowledged · 1 nullified · 1 not alleviated

REMEDIATION 02 July 31st, 2026

COMMIT `cf2b499154` ❖ `b069f20828`

1 alleviated · 1 partially alleviated · 1 acknowledged

FINAL August 1st, 2026

COMMIT `e0022c003c` ❖ `c02b898461`

1 alleviated

Audit Overview

We were tasked with performing an audit of the UltraYield codebase and in particular their updated vault implementations.

The codebase represents an updated implementation of the base UltraYield vaults with support for a cross-chain deployment via deterministic time-vesting oracle synchronization across CCIP as well as compliance and allowlist variants of the base vaults.

Over the course of the audit, we identified the absence of a pause state propagation across cross-chain deployments as well as a perpetual albeit minuscule value leak in the `BaseControlledAsyncRedeem` vault implementation that is inherited by all variations.

→ **We advise the UltraYield team to closely evaluate all minor-and-above findings identified in the report and promptly remediate them as well as consider all optimizational exhibits identified in the report.**

Post-Audit Conclusions

CONCLUSION 01 `d0f8fd98b7`

The UltraYield team iterated through all findings within the report and provided us with a revised commit hash to evaluate all exhibits on.

We evaluated all alleviations performed by UltraYield and have identified that certain exhibits have not been adequately dealt with. We advise the UltraYield team to revisit the following exhibits: `BCA-03M`, `UVT-02M`, `UVO-01C`

CONCLUSION 02 `cf2b499154`

The UltraYield team provided us with a follow-up commit to evaluate the alleviation / acknowledgement of the three aforementioned exhibits.

We evaluated the delta of this commit and consider `UVO-01C` to remain partially addressed, and we advise the UltraYield team to revisit it.

CONCLUSION 03 `e0022c003c`

The UltraYield team evaluated our recommendation on `UVO-01C` and proceeded with properly rectifying it.

As such, we consider all outputs of the audit report properly consumed by the UltraYield team.

Audit Synopsis

SEVERITY	IDENT.	ALLEV.	PARTIAL	ACKN.	NULLIF.
Major	0	0	0	0	0
Medium	1	0	0	1	0
Minor	7	3	0	3	1
Informational	25	22	0	3	0
Unknown	0	0	0	0	0

25 alleviated · 7 acknowledged · 1 nullified

During the audit, we filtered and validated a total of **1 findings utilizing static analysis** tools as well as identified a total of **32 findings during the manual review** of the codebase. We strongly recommend that any minor severity or higher findings are dealt with promptly prior to the project's launch as they can introduce potential misbehaviours of the system as well as exploits.

Report Chapters

The list below covers each segment of the audit in depth and links to the respective chapter of the report:

🎯 Scope →

A specification of the audit scope, including metadata such as the project's language and deployed network.

💻 Compilation →

Includes the compilation instructions that were followed as well as any issues that were identified by the compiler itself.

🔍 Static Analysis →

Briefly describes the output of the static analyzers of the project.

👁 Manual Review →

Briefly describes the output of the manual review of the project.

🖌 Code Style →

Describes the dedicated section of coding style and optimizational findings of the audit

CHAPTER 01

Scope

The audit engagement encompassed a specific list of contracts that were present in the commit hash of the repository that was in scope. The tables below detail certain meta-data about the target of the security assessment and a navigation chart is present at the end that links to the relevant findings per file.

Target

REPOSITORY	UltraYield/private_contracts
COMMIT	46cb108cf749d38dac24ad64c04a5d000bb8f585
LANGUAGE	Solidity
INTEGRATIONS	open-zeppelin
REVISIONS	46cb108cf7 d0f8fd98b7 cf2b499154 e0022c003c

Findings Per File

StableChainlinkRateProvider.sol	SCR-S SCR-C	3 →
AnyERC4626VaultRateProvider.sol	AER-M	1 →
BaseControlledAsyncRedeem.sol	BCA-M BCA-C	4 →
ChainlinkRateProvider.sol	CRP-M CRP-C	2 →
CompliantUltraVault.sol	CUV-M	1 →
UltraVault.sol	UVT-M UVT-C	3 →
UltraVaultEthAdapter.sol	UVE-M	1 →

UltraVaultFactory.sol	UVF-M	1	→
VaultPriceManager.sol	VPM-M VPM-C	4	→
PriceSyncMessage.sol	PSM-C	1	→
RedeemQueue.sol	RQE-C	1	→
RedeemQueueLib.sol	RQL-C	1	→
TimelockedUpgradeModule.sol	TUM-C	1	→
TimelockedUUPSUpgradeable.sol	TUU-C	1	→
UltraFrontendHelper.sol	UFH-C	2	→
UltraVaultOracle.sol	UVO-C	1	→
UltraVaultRateProvider.sol	UVR-C	5	→

Contracts Assessed

Carries findings No findings

AKS	AddressKeys.sol	AUS	AddressUpdates.sol	AUV	AllowlistUltraVault.sol
AER	AnyERC4626VaultRate... 1	BCA	BaseControlledAsyncR... 4	BVP	BaseVaultPriceManager.sol
CRP	ChainlinkRateProvider.... 2	CLB	ComplianceLib.sol	CUV	CompliantUltraVault.sol 1
CRR	ComposableRateProvider.s...	FPM	FixedPointMathLib.sol	PSM	PriceSyncMessage.sol 1
RQE	RedeemQueue.sol 1	RQL	RedeemQueueLib.sol 1	RVP	RemoteVaultPriceManager....
RSE	Roles.sol	SFR	SharedFreezeRegistry.sol	SCR	StableChainlinkRatePr... 3
TUM	TimelockedUpgradeM... 1	TUU	TimelockedUUPSUpgra... 1	UFH	UltraFrontendHelper.sol 2
UVT	UltraVault.sol 3	UVE	UltraVaultEthAdapter.s... 1	UVF	UltraVaultFactory.sol 1
UVO	UltraVaultOracle.sol 1	UVR	UltraVaultRateProvider.... 5	VPM	VaultPriceManager.sol 4

CHAPTER 02

Compilation

Includes the compilation instructions that were followed as well as any issues that were identified by the compiler itself.

PIPELINE	SOLC VERSION	COMPILER ERRORS
foundry	0.8.28	None

The project utilizes `foundry` as its development pipeline tool, containing an array of tests and scripts coded in Solidity.

To compile the project, the `build` command needs to be issued via the `forge` CLI tool:

BASH

```
$ forge build
```

The `forge` tool automatically selects Solidity version `0.8.28` based on the version specified within the `foundry.toml` file.

The project contains discrepancies with regards to the Solidity version used as the `pragma` statements of the contracts are open-ended (`⩾0.8.0`).

We advise them to be locked to `0.8.28` (`=0.8.28`), the same version utilized for our static analysis as well as optimizational review of the codebase.

During compilation with the `foundry` pipeline, no errors were identified that relate to the syntax or bytecode size of the contracts.

CHAPTER 03

Static Analysis

Briefly describes the output of the static analyzers of the project.

TOTAL DETECTIONS	FALSE POSITIVES	ACCEPTED	COMPILER ERRORS
63	61	2	0

The execution of our static analysis toolkit identified **63 potential issues** within the codebase of which **61 were ruled out to be false positives** or negligible findings.

The remaining **2 issues** were validated and grouped and formalized into the **1 exhibits** that follow:

StableChainlinkRateProvider.sol	SCR-S →

SCR-01S	Illegible Numeric Value Representations	INFO	ACK

STATIC ANALYSIS SCR-S

StableChainlinkRateProvider Static Analysis Findings

Contains all the findings that relate to static analysis on the contract codebase.

`src/oracles/StableChainlinkRateProvider.sol`

SCR-01S INFO Illegible Numeric Value Representations

TYPE	DISPOSITION	LOCATIONS
Code Style	• Acknowledged	I-1 StableChainlinkRateProvider.sol:L16 I-2 StableChainlinkRateProvider.sol:L17

DESCRIPTION

The linked representations of numeric literals are sub-optimally represented decreasing the legibility of the codebase.

AFFECTED CODE 2 instances — excerpt shown

`src/oracles/StableChainlinkRateProvider.sol` L16

```
16  uint256 internal constant BPS_DENOMINATOR = 10_000;
```

RECOMMENDATION

To properly illustrate each value's purpose, we advise the following guidelines to be followed. For values meant to depict fractions with a base of `1e18`, we advise fractions to be utilized directly (i.e. `1e17` becomes `0.1e18`) as they are supported. For values meant to represent a percentage base, we advise each value to utilize the underscore (`_`) separator to discern the percentage decimal (i.e. `10000` becomes `100_00`, `300` becomes `3_00` and so on). Finally, for large numeric values we simply advise the underscore character to be utilized again to represent them (i.e. `1000000` becomes `1_000_000`).

ACKNOWLEDGED d0f8fd98b7

The UltraYield team evaluated this exhibit but opted to acknowledge it in the current iteration of the codebase.

CHAPTER 04

Manual Review

Briefly describes the output of the manual review of the project.

A **thorough line-by-line review** was conducted on the codebase to identify potential malfunctions and vulnerabilities in UltraYield's updated vault implementations.

As the project at hand implements an EIP-4626 vault with asynchronous redemptions, intricate care was put into ensuring that the **flow of funds within the system conforms to the specifications and restrictions** laid forth within the protocol's specification.

We validated that **all state transitions of the system occur within sane criteria** and that all rudimentary formulas within the system execute as expected. We **pinpointed two significant vulnerabilities** within the system which could have had **moderate ramifications** to its overall operation; for more information, kindly consult the relevant medium-severity exhibits within the audit report.

Additionally, the system was investigated for any other commonly present attack vectors such as re-entrancy attacks, mathematical truncations, logical flaws and ERC / EIP standard inconsistencies. The documentation of the project was satisfactory to a great extent, containing extensive in-line documentation across the contracts.

A total of **32 findings** were identified over the course of the manual review of which **11 findings** concerned the behaviour and security of the system. The non-security related findings, such as optimizations, are included in the separate Code Style chapter.

The finding table below enumerates all these security / behavioural findings:

AnyERC4626VaultRateProvider.sol			AER-M →
AER-01M	Wrapped-Vault Rate Inherits Underlying Manipulate-ability	MINOR	ALLEV.

BaseControlledAsyncRedeem.sol

BCA-M →

BCA-01M	Incorrect Maximum Mint / Deposit Limitations	MINOR	ACK
BCA-02M	Unconventional Access Control Ruleset	MINOR	NULLIFIED
BCA-03M	Inexistent Handling of Truncated Values	MEDIUM	ACK

ChainlinkRateProvider.sol

CRP-M →

CRP-01M	Potentially Unsupported Assets	MINOR	ACK

CompliantUltraVault.sol

CUV-M →

CUV-01M	Inexistent Access of Fulfilled Proceeds	MINOR	ACK

UltraVault.sol

UVT-M →

UVT-01M	Absence of Access Control	INFO	ALLEV.
UVT-02M	Improper Rounding Direction of Instant Redemption Evaluation	MINOR	ALLEV.

UltraVaultEthAdapter.sol

UVE-M →

UVE-01M	Potential Out-of-Order Event Emission	INFO	ALLEV.

UltraVaultFactory.sol

UVF-M →

UVF-01M	Inexistent Validation of Non-Zero Version Key	INFO	ALLEV.

VaultPriceManager.sol

VPM-M →

VPM-01M	Inexistent Propagation of Pause State	MINOR	ALLEV.

MANUAL REVIEW AER-M

AnyERC4626VaultRateProvider Manual Review Findings

Contains all the findings that relate to manual review on the contract codebase.

src/oracles/AnyERC4626VaultRateProvider.sol

AER-01M

MINOR

Wrapped-Vault Rate Inherits Underlying Manipulate-ability

TYPE	DISPOSITION	LOCATION
Logical Fault	• Alleviated	AnyERC4626VaultRateProvider.sol:L51-L64

DESCRIPTION

The `AnyERC4626VaultRateProvider::convertToUnderlying` and `AnyERC4626VaultRateProvider::convertFromUnderlying` functions both rely on EIP-4626 functions that are themselves prone to manipulation via flash-loans as most EIP-4626 assets rely on spot evaluation of their checks and balances.

IMPACT

In contrast to other oracles of the system, the `AnyERC4626VaultRateProvider` contract is susceptible to on-chain manipulation without detailing this caveat.

AFFECTED CODE 1 instance

src/oracles/AnyERC4626VaultRateProvider.sol L46-L64

```
46  /// @inheritdoc IRateProvider
47  /// @notice Convert wrapped vault shares to its underlying asset amount
48  /// @param asset The asset to convert (must be the wrapped vault)
49  /// @param assets Amount of shares to convert
50  /// @return Amount in the vault's underlying asset
51  function convertToUnderlying(address asset, uint256 assets) external view re
52      require(asset == address(ASSET), InvalidAsset());
53      return ASSET.convertToAssets(assets);
```

```
54  }
55
56  /// @inheritdoc IRateProvider
57  /// @notice Convert underlying asset amount to wrapped vault shares
58  /// @param asset The asset to convert to (must be the wrapped vault)
59  /// @param underlyingAssets Amount of the vault's underlying asset to conver
60  /// @return Amount of shares
61  function convertFromUnderlying(address asset, uint256 underlyingAssets) exte
62      require(asset == address(ASSET), InvalidAsset());
63      return ASSET.convertToShares(underlyingAssets);
64  }
```

RECOMMENDATION

We advise this risk to be clearly outlined in the contract itself and to be taken into account when configuring assets in the Ultra Yield system.

● ALLEVIATED d0f8fd98b7

Proper documentation has been introduced outlining this risk, alleviating this exhibit.

MANUAL REVIEW BCA-M

BaseControlledAsyncRedeem Manual Review Findings

Contains all the findings that relate to manual review on the contract codebase.

`src/vaults/BaseControlledAsyncRedeem.sol`

BCA-01M MINOR Incorrect Maximum Mint / Deposit Limitations

TYPE	DISPOSITION	LOCATIONS
Standard Conformity	• Acknowledged	I-1 BaseControlledAsyncRedeem.sol:L609 I-2 BaseControlledAsyncRedeem.sol:L617

DESCRIPTION

The `BaseControlledAsyncRedeem::maxDeposit` and `BaseControlledAsyncRedeem::maxMint` functions attempt to be EIP-4626 compliant by taking into account `receiver` freezes as well as pause states, however, they do not properly take into account the actual total supply of the vault and thus the remaining balance that is available to be minted.

IMPACT

While unlikely to manifest in practice, the current `BaseControlledAsyncRedeem::maxDeposit` and `BaseControlledAsyncRedeem::maxMint` functions are not satisfying the EIP-4626 specification as they yield misleading "maximum" values in a non-empty state.

AFFECTED CODE 2 instances — excerpt shown

`src/vaults/BaseControlledAsyncRedeem.sol` L604-L618

```
604  /// @notice Get max assets for deposit
605  /// @return assets Maximum deposit amount
606  function maxDeposit(
607      address receiver
```

```
608 ) public view virtual override returns (uint256) {
609     return (paused() || !_canReceiveShares(receiver)) ? 0 : type(uint256).ma
610 }
611
612 /// @notice Get max shares for mint
613 /// @return shares Maximum mint amount
614 function maxMint(
615     address receiver
616 ) public view virtual override returns (uint256) {
617     return (paused() || !_canReceiveShares(receiver)) ? 0 : type(uint256).ma
618 }
```

RECOMMENDATION

We advise the calculations to be adjusted accordingly, ensuring the current total supply of the vault is taken into account.

● ACKNOWLEDGED d0f8fd98b7

The UltraYield team evaluated this exhibit and opted to acknowledge it as they do not consider the minuscule EIP conformity gained to outweigh the bytecode size impact.

BCA-02M MINOR

Unconventional Access Control Ruleset

TYPE	DISPOSITION	LOCATION
Logical Fault	• Nullified	BaseControlledAsyncRedeem.sol:L663-L679

DESCRIPTION

The `BaseControlledAsyncRedeem::requestRedeem` function results in an unconventional ACL pathway whereby a request redemption can be set for controller `B` via shares owned by owner `C` from account `A` that is authorized with an account `D` being able to control the `B` redemption request if they themselves are also a controller.

IMPACT

The current access control structure of redemption requests is unclear and can be exploited to trick users into creating incorrect requests that are consequently siphoned.

AFFECTED CODE 1 instance

src/vaults/BaseControlledAsyncRedeem.sol L711-L751

```
711  /// @notice Cancel redeem request for controller
712  /// @param _asset Asset
713  /// @param controller Controller address
714  /// @param receiver Share recipient
715  /// @return shares Amount of shares canceled
716  /// @dev Transfers pending shares back to receiver
717  function cancelRedeemRequest(
718      address _asset,
719      address controller,
720      address receiver
721  ) external returns (uint256 shares) {
722      shares = _cancelRedeemForAllPendingShares(_asset, controller, receiver);
723  }
724
725  /// @dev Internal function for canceling redeem requests for all pending sha
726  function _cancelRedeemForAllPendingShares(
727      address _asset,
728      address controller,
729      address receiver
730  ) internal returns (uint256 shares) {
```

```
731         shares = _getPendingRedeem(controller, _asset).shares;
732         _handleRedeemRequestCancelation(_asset, shares, controller, receiver);
733     }
734
735     /// @dev Internal function for canceling redeem requests
736     function _handleRedeemRequestCancelation(
737         address _asset,
738         uint256 shares,
739         address controller,
740         address receiver
741     ) internal virtual checkAccess(controller) {
742         // Consume pending redeem
743         require(shares != 0, NoPendingRedeem());
744         _consumePendingRedeem(controller, _asset, shares);
745
746         // Transfer pending shares from vault to receiver
747         _transfer(address(this), receiver, shares);
748
749         // Emit event
750         emit RedeemRequestCanceled(controller, receiver, shares);
751     }
```

RECOMMENDATION

We advise redemption requests to be allowed with a different controller only when `owner` is the caller themselves, and to enforce the `receiver` of a cancelled redemption request to be the `controller` if the caller is not the `controller` themselves.

● NULLIFIED d0f8fd98b7

The UltraYield team evaluated this exhibit and clarified that they wish to retain the current behaviour in place as they consider it intentional.

As the dual authorization system is known and acceptable, we consider this exhibit inapplicable.

BCA-03M MEDIUM

Inexistent Handling of Truncated Values

TYPE	DISPOSITION	LOCATIONS
Mathematical Operations	• Acknowledged	I-1 BaseControlledAsyncRedeem.sol:L405 I-2 BaseControlledAsyncRedeem.sol:L458 I-3 BaseControlledAsyncRedeem.sol:L827

DESCRIPTION

As a security precaution, all redemption calculation functions exposed by the `RedeemQueue` round in favour of the protocol.

As a result, user-initiated withdrawals can result in stranded funds in both share and asset denominations.

IMPACT

Over time, a perpetual accrual of both shares and units will result in the `BaseControlledAsyncRedeem` contract due to indivisible withdrawals being fulfilled.

AFFECTED CODE 3 instances — excerpt shown

src/vaults/BaseControlledAsyncRedeem.sol L447-L463

```
447  /// @dev Internal function for redeeming exact number of `shares` and withdr
448  /// @dev assets in `_asset` to `receiver`
449  function _redeemAsset(
450      address _asset,
451      uint256 shares,
452      address receiver,
453      address controller
454  ) internal checkAccess(controller) returns (uint256 assets) {
455      require(shares ≠ 0, NothingToRedeem());
456
457      // Calculate assets directly in asset units
458      assets = _calculateClaimableAssetsForShares(controller, _asset, shares);
459      require(assets ≠ 0, NothingToWithdraw());
460
461      // Execute withdraw
462      _performWithdraw(_asset, assets, shares, receiver, controller);
463  }
```

RECOMMENDATION

We advise these values to be properly handled by emptying out a redemption entry on the final operation that causes either value to go to `0`, ensuring that neither funds nor shares remain stranded in the contract.

RESPONSE `d0f8fd98b7`

The UltraYield team assessed this exhibit, proceeded to implement a fix, and requested a re-evaluation of the exhibit's severity.

In relation to the alleviation introduced, we consider it partial as the code does clean up lingering entries but does not actually yield the rounding-corrected values to the `RedeemQueueLib::consumeClaimableRedeem` caller, causing them to remain in the contract. These values should be yielded back and should overwrite whatever the request initially asked for.

Severity-wise we consider the exhibit to remain medium as the rounding issue is cascading (i.e. *each* partial withdraw can cause the truncation to occur *inadvertently* rather than in a deliberately self-inflicted manner) and directly affects user funds.

ACKNOWLEDGED `cf2b499154`

The UltraYield team evaluated this exhibit and, after careful consideration, has decided to acknowledge it *at the current state of the code*.

Specifically, they consider the rounding error to be cumulative yet immaterial in real monetary terms and a deliberate accumulation of the error to be self-inflicted harm.

As such, they have opted to document this behaviour and recommend withdrawals to be performed for the maximum amounts available to avoid such errors.

MANUAL REVIEW CRP-M

ChainlinkRateProvider Manual Review Findings

Contains all the findings that relate to manual review on the contract codebase.

src/oracles/ChainlinkRateProvider.sol

CRP-01M MINOR Potentially Unsupported Assets

TYPE	DISPOSITION	LOCATION
Standard Conformity	• Acknowledged	ChainlinkRateProvider.sol:L62, L63

DESCRIPTION

There are a few assets that Chainlink supports but that do not expose a `decimals` function as it is not part of the EIP-20 standard.

In such cases, most protocols default to `18` decimals or allow manual specification of the decimals.

IMPACT

Certain EIP-20 assets that have a Chainlink feed configured but do not support the extended EIP-20 `decimals` function signature will be incompatible with the UltraYield system.

AFFECTED CODE 1 instance

src/oracles/ChainlinkRateProvider.sol L44-L76

```
44  constructor(
45      address _underlyingAsset,
46      address _secondaryAsset,
47      address _underlyingFeed,
48      address _secondaryFeed,
49      uint256 _maxStaleness
50  ) {
51      require(_underlyingAsset ≠ address(0), ZeroAddress());
```

```
52      require(_secondaryAsset ≠ address(0), ZeroAddress());
53      require(_underlyingFeed ≠ address(0), ZeroAddress());
54      require(_secondaryFeed ≠ address(0), ZeroAddress());
55      require(_maxStaleness ≠ 0, ZeroMaxStaleness());
56
57      underlyingAsset = _underlyingAsset;
58      secondaryAsset = _secondaryAsset;
59      underlyingFeed = AggregatorV3Interface(_underlyingFeed);
60      secondaryFeed = AggregatorV3Interface(_secondaryFeed);
61
62      uint8 _underlyingDecimals = IERC20Metadata(_underlyingAsset).decimals();
63      uint8 _secondaryDecimals = IERC20Metadata(_secondaryAsset).decimals();
64      uint8 _underlyingFeedDecimals = underlyingFeed.decimals();
65      uint8 _secondaryFeedDecimals = secondaryFeed.decimals();
66
67      _validateDecimals(_underlyingDecimals);
68      _validateDecimals(_secondaryDecimals);
69      _validateDecimals(_underlyingFeedDecimals);
70      _validateDecimals(_secondaryFeedDecimals);
71      require(_underlyingFeedDecimals == _secondaryFeedDecimals, FeedDecimalsM
72
73      underlyingDecimals = _underlyingDecimals;
74      secondaryDecimals = _secondaryDecimals;
75      maxStaleness = _maxStaleness;
76  }
```

RECOMMENDATION

We advise proper support for legacy assets to be introduced to ensure the Chainlink system is compatible with all types of implementations.

● ACKNOWLEDGED d0f8fd98b7

The UltraYield team evaluated this exhibit and clarified that they wish to acknowledge it as they do not intend to support tokens without the `decimals` function exposed.

MANUAL REVIEW CUV-M

CompliantUltraVault Manual Review Findings

Contains all the findings that relate to manual review on the contract codebase.

src/vaults/CompliantUltraVault.sol

CUV-01M MINOR Inexistent Access of Fulfilled Proceeds

TYPE	DISPOSITION	LOCATION
Logical Fault	• Acknowledged	CompliantUltraVault.sol :L43-L48

DESCRIPTION

The `CompliantUltraVault` is unable to actually seize any fulfilled redemption requests of a malicious entity.

IMPACT

While the `CompliantUltraVault` is meant to be regulatory compliant, it is unable to affect any redemption requests that have been fulfilled including those in flight.

AFFECTED CODE 1 instance

src/vaults/CompliantUltraVault.sol L43-L48

```
43  /// @notice Burn `amount` shares from `target`'s wallet balance.
44  /// @dev Only affects `balanceOf(target)`; pending and claimable redeems are
45  function forceBurn(address target, uint256 amount) external onlyRole(COMPLIA
46      _burn(target, amount);
47      emit ForceBurned(target, amount);
48  }
```

RECOMMENDATION

We advise this capability to be introduced, further strengthening the compliancy of the `CompliantUltraVault` with regulatory bodies.

ACKNOWLEDGED d0f8fd98b7

The UltraYield team evaluated this exhibit and clarified that the current behaviour is intentional to simultaneously satisfy the compliance requirements of the contract whilst remaining under the bytecode size limit.

As such, we consider this exhibit safely acknowledged.

MANUAL REVIEW UVT-M

UltraVault Manual Review Findings

Contains all the findings that relate to manual review on the contract codebase.

src/vaults/UltraVault.sol

UVT-01M INFO Absence of Access Control

TYPE	DISPOSITION	LOCATION
Standard Conformity	• Alleviated	UltraVault.sol:L136-L164

DESCRIPTION

The `UltraVault::initializeV2` reinitialization mechanism does not employ any authorization and thus can be front-ran.

AFFECTED CODE 1 instance

src/vaults/UltraVault.sol L136-L164

```
136  /// @notice V1 → V2 migration hook
137  /// @dev Copies only the live V1 contract addresses into the V2 address regi
138  /// @dev Pending V1 address-update proposals are intentionally not migrated
139  /// @param _instantRedeemExitpoint Address for the new instant redeem exitpo
140  /// @param _upgradeModule Address of the external timelocked upgrade module
141  /// @param _freezeRegistry Address of the shared freeze registry (optional;
142  function initializeV2(
143      address _instantRedeemExitpoint,
144      address _upgradeModule,
145      address _freezeRegistry
146  ) external reinitializer(2) {
147      UltraVaultStorage storage $ = _getUltraVaultStorage();
148      // Guard: only a genuine V1 proxy carries a live address in the deprecat
149      require($._fundsHolder ≠ address(0), NotMigratable());
150
151      _migrateBaseControlledAsyncRedeemStateFromV1();
152
```

```
153     _migrateAddress(FUNDS_HOLDER_KEY, $._fundsHolder);
154     _migrateAddress(ORACLE_KEY, address($._oracle));
155     _migrateAddress(FREEZE_REGISTRY_KEY, _freezeRegistry);
156     _initAddress(INSTANT_REDEEM_EXITPOINT_KEY, _instantRedeemExitpoint);
157     _initAddress(UPGRADE_MODULE_KEY, _upgradeModule);
158
159     // Rescale the V1 high-water mark into V2's 1e18 price space. In V1 the
160     // 10**decimals() share-value units; V2 compares it against the 1e18-sca
161     $.fees.highwaterMark *= 10 ** (18 - uint256(decimals()));
162
163     if (!paused()) _pause();
164 }
```

RECOMMENDATION

We advise the function to either apply access control or to be documented as transaction ordering sensitive, either of which we consider an adequate alleviation of this exhibit.

ALLEVIATED `d0f8fd98b7`

A disclaimer was introduced to the code outlining that the function is ordering-sensitive and must be invoked the moment the contract is deployed, addressing this exhibit.

UVT-02M

MINOR

Improper Rounding Direction of Instant Redemption Evaluation

TYPE	DISPOSITION	LOCATION
Mathematical Operations	• Alleviated	UltraVault.sol:L344

DESCRIPTION

The `UltraVault::maxInstantRedeem` function overestimates the maximum instant redemption due to rounding the underlying assets that the instant liquidity has upward, causing incorrect evaluations when the asset has higher decimals than its base (the underlying).

IMPACT

A redemption at the exact `UltraVault::maxInstantRedeem` boundary may fail when the asset's decimals are higher than the underlying's decimals due to an improper rounding direction.

AFFECTED CODE 1 instance

src/vaults/UltraVault.sol L327-L347

```
327  /// @notice Returns the maximum shares a controller can instant-redeem for a
328  /// @dev Considers both the controller's share balance and available liquidi
329  ///      Liquidity bounds the gross payout (netAssets + fee are both pulled
330  ///      so the shares are derived from liquidity directly and capped at con
331  /// @param _asset The asset address to redeem for
332  /// @param controller The address whose shares would be redeemed
333  /// @return shares The maximum redeemable shares
334  function maxInstantRedeem(
335      address _asset,
336      address controller
337  ) external view returns (uint256 shares) {
338      if (!_canInstantRedeem(controller)) return 0;
339      uint256 liquidity = getLiquidity(_asset);
340      uint256 shareBalance = balanceOf(controller);
341      if (liquidity == 0 || shareBalance == 0) {
342          return 0;
343      }
344      uint256 maxUnderlyingAssets = convertToUnderlying(_asset, liquidity, Adj
345      uint256 maxShares = convertToShares(maxUnderlyingAssets);
346      return maxShares < shareBalance ? maxShares : shareBalance;
```

```
347 }
```

RECOMMENDATION

As rounding down will affect the other decimal cases negatively, we advise this caveat to be outlined in the documentation of the function or to be accounted for in the calculation specifically for the decimal case outlined.

RESPONSE `d0f8fd98b7`

The UltraYield team seems to have provided a response that relates to `UVT-01M`, rendering this exhibit to remain open.

ALLEVIATED `cf2b499154`

The UltraYield team opted to implement the first of our two recommended approaches, updating the documentation of the function to explicitly call out this caveat for would-be integrators.

As such, we consider this exhibit fully addressed.

MANUAL REVIEW UVE-M

UltraVaultEthAdapter Manual Review Findings

Contains all the findings that relate to manual review on the contract codebase.

src/periphery/UltraVaultEthAdapter.sol

UVE-01M INFO Potential Out-of-Order Event Emission

TYPE	DISPOSITION	LOCATION
Logical Fault	• Alleviated	UltraVaultEthAdapter.sol:L146

DESCRIPTION

The `EthWithdraw` event may be emitted out-of-order if a `receiver` re-enters the contract.

AFFECTED CODE 1 instance

src/periphery/UltraVaultEthAdapter.sol L125-L147

```
125  /// @notice Redeem shares for native ETH from fulfilled requests
126  /// @param shares Amount of shares to redeem
127  /// @param receiver Address to receive the ETH
128  /// @return ethAmount Amount of ETH received
129  /// @dev Works with fulfilled redemption requests, unwraps WETH to ETH
130  function redeemEth(
131      uint256 shares,
132      address receiver
133  ) public whenNotPaused returns (uint256 ethAmount) {
134      require(receiver ≠ address(0), ZeroAddress());
135
136      // Redeem WETH from vault to this adapter
137      address controller = msg.sender;
138      ethAmount = vault.redeemAsset(address(weth), shares, address(this), cont
139
140      // Unwrap WETH to ETH
```

```
141     weth.withdraw(ethAmount);
142     // Send ETH to receiver
143     (bool success, ) = receiver.call{value: ethAmount}("");
144     require(success, EthTransferFailed());
145
146     emit EthWithdraw(msg.sender, receiver, controller, ethAmount, shares);
147 }
```

RECOMMENDATION

We advise the `event` to be emitted prior to the external call, ensuring that events are always emitted in the correct order.

● ALLEVIATED `d0f8fd98b7`

The `EthWithdraw` event is now emitted prior to the external fund transfer, addressing this exhibit.

MANUAL REVIEW UVF-M

UltraVaultFactory Manual Review Findings

Contains all the findings that relate to manual review on the contract codebase.

src/factory/UltraVaultFactory.sol

UVF-01M INFO Inexistent Validation of Non-Zero Version Key

TYPE	DISPOSITION	LOCATION
Input Sanitization	• Alleviated	UltraVaultFactory.sol:L95

DESCRIPTION

While the version key is a hash rendering the possibility of a zero-value key minuscule, the system will misbehave in such a case and permit a re-registration of the same implementation.

AFFECTED CODE 1 instance

src/factory/UltraVaultFactory.sol L84-L109

```
84  /// @inheritdoc IUltraVaultFactory
85  function registerVersion(
86      Version calldata version,
87      address implementation,
88      bytes4 initSelector
89  ) external onlyRole(DEFAULT_ADMIN_ROLE) {
90      require(implementation ≠ address(0), ZeroImplementation());
91      require(implementation.code.length ≠ 0, ImplementationNotContract(imple
92      require(initSelector ≠ bytes4(0), ZeroInitSelector());
93
94      UltraVaultFactoryStorage storage $ = _getUltraVaultFactoryStorage();
95      bytes32 key = _getVersionKey(version);
96      require($.versions[key].implementation == address(0), VersionAlreadyRegi
97      require($.versionKeyByImpl[implementation] == bytes32(0), Implementation
98
```

```
99      $.versions[key] = VersionInfo({
100         version: version,
101         implementation: implementation,
102         initSelector: initSelector,
103         disabled: false
104     });
105     $.versionKeyByImpl[implementation] = key;
106     $.versionKeys.push(key);
107
108     emit VersionRegistered(key, implementation, initSelector, version);
109 }
```

RECOMMENDATION

We advise the code to ensure that the key fetched is non-zero, preventing the registration of incorrect version keys.

● ALLEVIATED d0f8fd98b7

A `require` check was introduced ensuring a non-zero version key is generated, alleviating this exhibit.

MANUAL REVIEW VPM-M

VaultPriceManager Manual Review Findings

Contains all the findings that relate to manual review on the contract codebase.

src/oracles/VaultPriceManager.sol

VPM-01M MINOR Inexistent Propagation of Pause State

TYPE	DISPOSITION	LOCATION
Logical Fault	• Alleviated	VaultPriceManager.sol:L109, L119

DESCRIPTION

The main-chain price updates through the `VaultPriceManager` may result in the vault becoming paused, however, no such behaviour is observed in the target chains through the `RemoteVaultPriceManager`.

IMPACT

A pause state trigger in the main chain will not be propagated to the remote chains thereby exacerbating a sharp market downturn / upturn by permitting arbitrage in satellite chains.

AFFECTED CODE 1 instance

src/oracles/VaultPriceManager.sol L104-L121

```
104  /// @notice Schedule an instant update and broadcast it. Activates at `now +
105  /// @param failOnPause If true, revert instead of pausing when `newPrice` br
106  function instantPriceUpdate(uint256 newPrice, bool failOnPause) external onl
107      uint256 startTimestamp = block.timestamp + delay;
108      // startPrice is unused for an instant update; reuse newPrice.
109      _applyUpdate(newPrice, newPrice, 0, startTimestamp, failOnPause);
110      _broadcastUpdate(newPrice, newPrice, 0, startTimestamp);
111  }
112
```

```
113 /// @notice Schedule a linear-vested update and broadcast it. Vesting begins
114 ///         `now + delay`; the start price is projected to that instant for
115 /// @param failOnPause If true, revert instead of pausing when `newPrice` br
116 function linearPriceUpdate(uint256 newPrice, uint256 duration, bool failOnPa
117     uint256 startTimestamp = block.timestamp + delay;
118     uint256 startPrice = oracle.sharePriceAt(startTimestamp);
119     _applyUpdate(startPrice, newPrice, duration, startTimestamp, failOnPause
120     _broadcastUpdate(startPrice, newPrice, duration, startTimestamp);
121 }
```

RECOMMENDATION

We advise the code to handle the pause state differently, broadcasting it alongside the price update and ensuring `RemoteVaultPriceManager` destinations accept a pause state regardless of the nonce of the message to support out-of-order pause processing.

● ALLEVIATED d0f8fd98b7

The UltraYield team proceeded to address this exhibit by ensuring pauses are propagated regardless of whether the local pause guard is triggered on the satellite chains.

As such, we consider this exhibit fully alleviated.

CHAPTER 05

Code Style

Describes the dedicated section of coding style and optimizational findings of the audit

During the manual portion of the audit, we identified **21 optimizations** that can be applied to the codebase that will decrease the operational cost associated with the execution of a particular function and generally ensure that the project complies with the latest best practices and standards in Solidity.

Additionally, this section of the audit contains any opinionated adjustments we believe the code should make to make it more legible as well as truer to its purpose.

These optimizations are enumerated below:

BaseControlledAsyncRedeem.sol	BCA-C →
BCA-01C	Generic Typographic Mistakes	INFO	ALLEV.

ChainlinkRateProvider.sol	CRP-C →
CRP-01C	Inefficient Safe Arithmetic	INFO	ALLEV.

PriceSyncMessage.sol	PSM-C →
PSM-01C	Improper Ordering of Variables	INFO	ACK

RedeemQueue.sol	RQE-C →
RQE-01C	Unused Import	INFO	ALLEV.

RedeemQueueLib.sol	RQL-C →
RQL-01C	Inefficient Safe Arithmetic	INFO	ALLEV.

StableChainlinkRateProvider.sol

SCR-C →

SCR-01C	Inefficient Event Emission	INFO	ALLEV.
SCR-02C	Inefficient Safe Arithmetic	INFO	ALLEV.

TimelockedUpgradeModule.sol

TUM-C →

TUM-01C	Optimization of Storage Footprint	INFO	ALLEV.

TimelockedUUPSUpgradeable.sol

TUU-C →

TUU-01C	Optimization of Storage Footprint	INFO	ALLEV.

UltraFrontendHelper.sol

UFH-C →

UFH-01C	Inefficient Logic Split of Length Assessment	INFO	ALLEV.
UFH-02C	Inefficient Safe Arithmetic	INFO	ALLEV.

UltraVault.sol

UVT-C →

UVT-01C	Inefficient Safe Arithmetic	INFO	ALLEV.

UltraVaultOracle.sol

UVO-C →

UVO-01C	Inefficient Safe Arithmetic	INFO	ALLEV.

UltraVaultRateProvider.sol

UVR-C →

UVR-01C	Inefficient Event Emissions	INFO	ALLEV.
UVR-02C	Inefficient Initial Addition	INFO	ALLEV.
UVR-03C	Inefficient Safe Arithmetic	INFO	ALLEV.
UVR-04C	Potentially Redundant Shift Operation	INFO	ALLEV.
UVR-05C	Unreachable Code	INFO	ACK

VaultPriceManager.sol

VPM-C →

VPM-01C	Inefficient Event Emission	INFO	ALLEV.
VPM-02C	Inefficient `mapping` Lookups	INFO	ALLEV.
VPM-03C	Potentially Redundant Shift Operation	INFO	ALLEV.

CODE STYLE BCA-C

BaseControlledAsyncRedeem Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

`src/vaults/BaseControlledAsyncRedeem.sol`

BCA-01C INFO Generic Typographic Mistakes

TYPE	DISPOSITION	LOCATIONS
Code Style	• Alleviated	I-1 BaseControlledAsyncRedeem.sol:L595 I-2 BaseControlledAsyncRedeem.sol:L708, L736

DESCRIPTION

The referenced lines contain typographical mistakes (i.e. `private` variable without an underscore prefix) or generic documentational errors (i.e. copy-paste) that should be corrected.

AFFECTED CODE 2 instances — excerpt shown

src/vaults/BaseControlledAsyncRedeem.sol L595

```
595  // Deposti & Withdraw Limits //
```

RECOMMENDATION

We advise them to be corrected enhancing the legibility of the codebase.

• ALLEVIATED d0f8fd98b7

All typographic mistakes have been addressed.

CODE STYLE CRP-C

ChainlinkRateProvider Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

src/oracles/ChainlinkRateProvider.sol

CRP-01C INFO Inefficient Safe Arithmetic

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Alleviated	ChainlinkRateProvider.sol:L139

DESCRIPTION

The referenced time delta can be computed using `unchecked` arithmetic as it is guaranteed to be safe by the preceding `require` check.

AFFECTED CODE 1 instance

src/oracles/ChainlinkRateProvider.sol L138-L139

```
138  require(updatedAt ⩽ block.timestamp, InvalidFeedAnswer());
139  require(block.timestamp - updatedAt ⩽ maxStaleness, StaleFeed());
```

RECOMMENDATION

We advise this to be done so, optimizing the code's gas cost.

• ALLEVIATED d0f8fd98b7

The referenced subtraction has been safely wrapped in an `unchecked` code block, optimizing the code's gas cost.

CODE STYLE PSM-C

PriceSyncMessage Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

src/crosschain/PriceSyncMessage.sol

PSM-01C INFO Improper Ordering of Variables

TYPE	DISPOSITION	LOCATION
Code Style	• Acknowledged	PriceSyncMessage.sol:L9 -L15

DESCRIPTION

The `PriceSyncMessage` currently leaves several empty gaps across both of the storage slots it occupies.

AFFECTED CODE 1 instance

src/crosschain/PriceSyncMessage.sol L4-L15

```
4  /// @title PriceSyncMessage
5  /// @notice Wire format for a cross-chain price update broadcast.
6  /// @dev    Sent by the main-chain `VaultPriceManager` to every active desti
7  ///         and consumed by `RemoteVaultPriceManager`. Both sides import thi
8  ///         struct + lib so the encoding stays in lockstep.
9  struct PriceSyncMessage {
10     uint64 nonce;
11     uint88 startPrice;
12     uint88 targetPrice;
13     uint40 vestingDuration;
14     uint40 startTimestamp;
15 }
```

RECOMMENDATION

We advise the `uint64 nonce` to be relocated after the `startTimestamp`, permitting the `nonce` to be increased in accuracy if wished so without any gas impact.

ACKNOWLEDGED d0f8fd98b7

The UltraYield team assessed this exhibit and considers the required infrastructure changes for the `struct` to be updated to be significant, opting instead to acknowledge it.

To note, even if the structure is not utilized for storage a custom encoding solution can be introduced to minimize the cross-chain ABI payload size and thus overall gas across the systems.

CODE STYLE RQE-C

RedeemQueue Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

`src/vaults/accounting/RedeemQueue.sol`

RQE-01C INFO Unused Import

TYPE	DISPOSITION	LOCATION
Code Style	• Alleviated	RedeemQueue.sol:L4

DESCRIPTION

The `UUPSUpgradeable` contract is imported in the `RedeemQueue` file but remains unused.

AFFECTED CODE 1 instance

`src/vaults/accounting/RedeemQueue.sol` L4

```
4  import { UUPSUpgradeable } from "@openzeppelin/contracts-upgradeable/proxy/u
```

RECOMMENDATION

We advise the import to be removed.

• ALLEVIATED d0f8fd98b7

The unused import has been properly removed.

CODE STYLE RQL-C

RedeemQueueLib Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

src/utils/RedeemQueueLib.sol

RQL-01C INFO Inefficient Safe Arithmetic

TYPE	DISPOSITION	LOCATIONS
Gas Optimization	• Alleviated	I-1 RedeemQueueLib.sol:L144 I-2 RedeemQueueLib.sol:L180 I-3 RedeemQueueLib.sol:L181

DESCRIPTION

The referenced subtractions are guaranteed to be performed safely based on their preceding `require` checks.

AFFECTED CODE 3 instances — excerpt shown

src/utils/RedeemQueueLib.sol L171-L182

```
171 function consumeClaimableRedeem(
172     address user,
173     address token,
174     uint256 assets,
175     uint256 shares
176 ) external {
177     ClaimableRedeemSlot storage redeem = _getStorage().claimableRedeems[user
178     require(assets ⩽ redeem.assets, IRedeemQueueErrors.InsufficientClaimabl
179     require(shares ⩽ redeem.shares, IRedeemQueueErrors.InsufficientClaimabl
180     redeem.assets -= uint128(assets);
181     redeem.shares -= uint128(shares);
182 }
```

RECOMMENDATION

We advise them to be wrapped in an `unchecked` code block, optimizing the code's gas cost.

ALLEVIATED `d0f8fd98b7`

The referenced subtractions have been safely wrapped in an `unchecked` code block, optimizing the code's gas cost.

CODE STYLE SCR-C

StableChainlinkRateProvider Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

src/oracles/StableChainlinkRateProvider.sol

SCR-01C INFO Inefficient Event Emission

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Alleviated	StableChainlinkRateProvider.sol:L85, L87

DESCRIPTION

The referenced event emission can be optimized by emitting it before the variable is mutated.

AFFECTED CODE 1 instance

src/oracles/StableChainlinkRateProvider.sol L80-L88

```
80  function _setMaxDeviationBps(uint256 newMaxDeviationBps) internal {
81      require(
82          newMaxDeviationBps ≠ 0 && newMaxDeviationBps ⩽ MAX_DEVIATION_BPS,
83          InvalidMaxDeviationBps()
84      );
85      uint256 oldValue = maxDeviationBps;
86      maxDeviationBps = newMaxDeviationBps;
87      emit MaxDeviationBpsUpdated(oldValue, newMaxDeviationBps);
88  }
```

RECOMMENDATION

We advise this to be done so, utilizing the `maxDeviationBps` storage entry directly and avoiding the memory reservation of `oldValue`.

● ALLEVIATED d0f8fd98b7

The inefficient event emission has been optimized as advised.

SCR-02C INFO

Inefficient Safe Arithmetic

TYPE	DISPOSITION	LOCATIONS
Gas Optimization	● Alleviated	I-1 StableChainlinkRateProvider.sol:L75 I-2 StableChainlinkRateProvider.sol:L76

DESCRIPTION

The referenced subtractions can be safely performed regardless of the variable values due to the ternary conditional.

AFFECTED CODE 2 instances — excerpt shown

src/oracles/StableChainlinkRateProvider.sol L74-L76

```
74  uint256 diff = assetPrice > underlyingPrice
75      ? assetPrice - underlyingPrice
76      : underlyingPrice - assetPrice;
```

RECOMMENDATION

We advise them to be wrapped in an `unchecked` code block, optimizing the code's gas cost.

● ALLEVIATED d0f8fd98b7

The referenced subtractions have been safely wrapped in an `unchecked` code block, optimizing the code's gas cost.

CODE STYLE TUM-C

TimelockedUpgradeModule Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

src/utils/TimelockedUpgradeModule.sol

TUM-01C INFO Optimization of Storage Footprint

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Alleviated	TimelockedUpgradeModule.sol:L71-L78

DESCRIPTION

The `PendingUpgrade` data structure can be optimized by relocating the `executionTime` variable after the `implementation` and setting it to a value of `uint96` which is more than capable of storing a Unix timestamp.

AFFECTED CODE 1 instance

src/utils/TimelockedUpgradeModule.sol L70-L78

```
70  /// @dev Represents the single pending upgrade
71  struct PendingUpgrade {
72      // The proposed implementation address
73      address implementation;
74      // Hash of the data to execute a call after the upgrade
75      bytes32 dataHash;
76      // Earliest when the upgrade can be executed (0 if not proposed)
77      uint256 executionTime;
78  }
```

RECOMMENDATION

We advise this to be done so, reducing the gas cost required to read and write pending upgrades by a full storage slot.

ALLEVIATED d0f8fd98b7

The structure and surrounding usage flows have been updated to utilize a `uint96` data entry instead, addressing this exhibit.

CODE STYLE TUU-C

TimelockedUUPSUpgradeable Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

`src/utils/TimelockedUUPSUpgradeable.sol`

TUU-01C INFO Optimization of Storage Footprint

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Alleviated	TimelockedUUPSUpgradeable.sol:L58-L66

DESCRIPTION

The `PendingUpgrade` data structure can be optimized by relocating the `executionTime` variable after the `implementation` and setting it to a value of `uint96` which is more than capable of storing a Unix timestamp.

AFFECTED CODE 1 instance

src/utils/TimelockedUUPSUpgradeable.sol L58-L66

```
58  /// @dev Represents the single pending upgrade
59  struct PendingUpgrade {
60      // The proposed implementation address
61      address implementation;
62      // Hash of the data to execute a call after the upgrade
63      bytes32 dataHash;
64      // Earliest when the upgrade can be executed (0 if not proposed)
65      uint256 executionTime;
66  }
```

RECOMMENDATION

We advise this to be done so, reducing the gas cost required to read and write pending upgrades by a full storage slot.

ALLEVIATED d0f8fd98b7

The structure and surrounding usage flows have been updated to utilize a `uint96` data entry instead, addressing this exhibit.

CODE STYLE UFH-C

UltraFrontendHelper Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

src/periphery/UltraFrontendHelper.sol

UFH-01C INFO Inefficient Logic Split of Length Assessment

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Alleviated	UltraFrontendHelper.sol :L67-L77, L81-L99

DESCRIPTION

The `UltraFrontendHelper::collectPendingRedeems` function will inefficiently iterate the full combination of assets and users twice; once to assess the appropriate array size and once to populate it.

AFFECTED CODE 1 instance

src/periphery/UltraFrontendHelper.sol L60-L101

```
60  /// @inheritdoc IUltraFrontendHelper
61  function collectPendingRedeems(address[] calldata assets, address[] calldata
62      // 1. Validate assets
63      IUltraVaultRateProvider rateProvider = vault.rateProvider();
64      for (uint256 i = 0; i < assets.length; i++) {
65          require(rateProvider.isSupported(assets[i]), "UNSUPPORTED_ASSET");
66      }
67      // 2. Count number of redeems
68      uint256 totalRedeems = 0;
69      for (uint256 i = 0; i < users.length; i++) {
70          address user = users[i];
71          for (uint256 j = 0; j < assets.length; j++) {
72              uint256 shares = vault.getPendingRedeem(assets[j], user).shares;
73              if (shares > 0) {
74                  totalRedeems++;
75              }
76          }
77      }
```

```
78      if (totalRedeems == 0) {
79          return redeems;
80      }
81      // 3. Build the response with redeems
82      redeems = new Redeem[](totalRedeems);
83      uint256 index = 0;
84      for (uint256 i = 0; i < users.length; i++) {
85          address user = users[i];
86          for (uint256 j = 0; j < assets.length; j++) {
87              address asset = assets[j];
88              PendingRedeem memory redeem = vault.getPendingRedeem(asset, user
89              if (redeem.shares > 0) {
90                  redeems[index] = Redeem({
91                      user: user,
92                      asset: asset,
93                      shares: redeem.shares,
94                      requestTime: redeem.requestTime
95                  });
96                  index++;
97              }
98          }
99      }
100     return redeems;
101 }
```

RECOMMENDATION

We advise the array size to be set to the maximum size posssible and to be resized after the actual count of redemptions is assessed, reducing the function's gas cost significantly and thus permitting it to operate under higher asset-user combinations safely.

● ALLEVIATED `d0f8fd98b7`

The array code was updated per our recommendation, shrinking the final array instead of looking ahead redundantly to assess its required size.

UFH-02C INFO

Inefficient Safe Arithmetic

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Alleviated	UltraFrontendHelper.sol :L57

DESCRIPTION

The referenced subtraction is guaranteed to be safe based on the withdrawal fee restrictions within the `UltraVault`.

AFFECTED CODE 1 instance

src/periphery/UltraFrontendHelper.sol L57

```
57  return assets - withdrawalFee;
```

RECOMMENDATION

We advise it to be performed in an `unchecked` code block, optimizing its gas cost.

• ALLEVIATED d0f8fd98b7

The referenced subtraction has been safely wrapped in an `unchecked` code block, optimizing the code's gas cost.

CODE STYLE UVT-C

UltraVault Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

src/vaults/UltraVault.sol

UVT-01C INFO Inefficient Safe Arithmetic

TYPE	DISPOSITION	LOCATIONS
Gas Optimization	Alleviated	I-1 UltraVault.sol:L463 I-2 UltraVault.sol:L676 I-3 UltraVault.sol:L698

DESCRIPTION

The referenced subtractions can be safely performed regardless of the variable values due to the ternary conditional.

AFFECTED CODE 3 instances — excerpt shown

src/vaults/UltraVault.sol L676

```
676  uint256 timePassed = block.timestamp - fees.lastUpdateTimestamp;
```

RECOMMENDATION

We advise them to be wrapped in an `unchecked` code block, optimizing the code's gas cost.

ALLEVIATED d0f8fd98b7

The referenced subtractions have been safely wrapped in an `unchecked` code block, optimizing the code's gas cost.

CODE STYLE UVO-C

UltraVaultOracle Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

src/oracles/UltraVaultOracle.sol

UVO-01C INFO Inefficient Safe Arithmetic

TYPE	DISPOSITION	LOCATIONS
Gas Optimization	• Alleviated	I-1 UltraVaultOracle.sol :L75 I-2 UltraVaultOracle.sol :L191 I-3 UltraVaultOracle.sol :L192

DESCRIPTION

The referenced subtractions are all guaranteed to be safe due to the various time-based and price-based constraints in the codebase.

AFFECTED CODE 3 instances — excerpt shown

src/oracles/UltraVaultOracle.sol L191

```
191  uint256 diff = uint256(u.targetPrice) - uint256(u.startPrice);
```

RECOMMENDATION

We advise them to be wrapped in `unchecked` code blocks, optimizing their gas cost.

• RESPONSE d0f8fd98b7

The `unchecked` code block introduced for the I-2 and I-3 instances was improperly expanded to cover the multiplication as well, rendering this exhibit improperly addressed.

RESPONSE `cf2b499154`

The UltraYield team did not revert the changes introduced by the previous commit which cover the multiplication nor did they adjust the `unchecked` block to no longer cover it, rendering the exhibit to remain partially addressed.

ALLEVIATED `e0022c003c`

The `unchecked` code block of I-2 and I-3 was properly rectified to only cover the safe-to-perform subtractions, addressing this exhibit in full.

CODE STYLE UVR-C

UltraVaultRateProvider Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

src/oracles/UltraVaultRateProvider.sol

UVR-01C INFO Inefficient Event Emissions

TYPE	DISPOSITION	LOCATIONS
Gas Optimization	• Alleviated	I-1 UltraVaultRateProvider.sol:L122, L126 I-2 UltraVaultRateProvider.sol:L138, L142

DESCRIPTION

The referenced event emissions can be optimized by emitting each before their respective variable is mutated.

AFFECTED CODE 2 instances — excerpt shown

src/oracles/UltraVaultRateProvider.sol L110-L127

```
110  /// @inheritdoc IUltraVaultRateProvider
111  function updateRateProvider(address asset, bool isPegged, address newRatePro
112      // Checks
113      require(asset ≠ baseAsset, CannotUpdateBaseAsset());
114      require(isSupported(asset), AssetNotSupported());
115      bool rateProviderIsValid = isPegged
116          ? newRateProvider == address(0)
117          : newRateProvider ≠ address(0);
118      require(rateProviderIsValid, InvalidRateProvider());
119
120      // Update storage
121      AssetSlot storage slot = supportedAssets[asset];
122      address oldRateProvider = slot.rateProvider;
123      slot.rateProvider = newRateProvider;
124
```

```
125         // Emit event
126         emit RateProviderUpdated(asset, oldRateProvider, newRateProvider);
127     }
```

RECOMMENDATION

We advise this to be done so, utilizing the pre-mutation storage entry directly and avoiding the memory reservation of a local variable.

ALLEVIATED d0f8fd98b7

The inefficient event emissions have been optimized as advised.

UVR-02C INFO

Inefficient Initial Addition

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Alleviated	UltraVaultRateProvider.sol:L58

DESCRIPTION

The `baseAsset` addition will perform several checks redundantly.

AFFECTED CODE 1 instance

src/oracles/UltraVaultRateProvider.sol L55-L59

```
55  constructor(address _owner, address _baseAsset) Ownable(_owner) {
56      baseAsset = _baseAsset;
57      baseDecimals = IERC20Metadata(_baseAsset).decimals();
58      _addAsset(_baseAsset, true, address(0), 0);
59  }
```

RECOMMENDATION

We advise the initial addition to be done manually as the system's state is guaranteed to be empty at that state and the checks around pegged assets are redundant.

• ALLEVIATED d0f8fd98b7

The initial addition was inlined to optimize the code's gas cost per our recommendation.

UVR-03C INFO

Inefficient Safe Arithmetic

TYPE	DISPOSITION	LOCATIONS
Gas Optimization	• Alleviated	I-1 UltraVaultRateProvider.sol:L240 I-2 UltraVaultRateProvider.sol:L244

DESCRIPTION

The referenced subtractions are guaranteed to be safe due to the `MAX_ADJUSTMENT_RATE` restriction.

AFFECTED CODE 2 instances — excerpt shown

src/oracles/UltraVaultRateProvider.sol L240

```
240  return Math.mulDiv(amount, SCALE - uint256(adjustmentRate), SCALE, Math.Roun
```

RECOMMENDATION

We advise them to be performed in an `unchecked` code block, optimizing the code's gas cost.

• ALLEVIATED d0f8fd98b7

The referenced subtractions have been safely wrapped in an `unchecked` code block, optimizing the code's gas cost.

UVR-04C INFO

Potentially Redundant Shift Operation

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Alleviated	UltraVaultRateProvider.sol:L100

DESCRIPTION

The `UltraVaultRateProvider::removeAsset` will redundantly assign the last entry to itself if the `for` loop exhausts all entries.

AFFECTED CODE 1 instance

src/oracles/UltraVaultRateProvider.sol L90-L108

```
90  /// @inheritdoc IUltraVaultRateProvider
91  function removeAsset(address asset) external onlyOwner {
92      // Checks
93      require(asset ≠ baseAsset, CannotUpdateBaseAsset());
94      require(isSupported(asset), AssetNotSupported());
95
96      // Update storage
97      delete supportedAssets[asset];
98      for (uint256 i = 0; i < assetList.length; i++) {
99          if (assetList[i] == asset) {
100             assetList[i] = assetList[assetList.length - 1];
101             assetList.pop();
102             break;
103         }
104     }
105
106     // Emit event
107     emit AssetRemoved(asset);
108 }
```

RECOMMENDATION

We advise the `for` loop to run until the `assetList.length - 1` value, and the `pop` instruction to be relocated after the `for` loop optimizing its cost.

ALLEVIATED d0f8fd98b7

The `for` lookup iteration and associated pop instruction were optimized per our recommendation.

UVR-05C INFO

Unreachable Code

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Acknowledged	UltraVaultRateProvider.sol:L246

DESCRIPTION

The `Adjustment` enum supports two states and the `UltraVaultRateProvider::_applyAdjustment` code will issue a `return` statement on both of the enum's states, rendering the final statement unreachable.

AFFECTED CODE 1 instance

src/oracles/UltraVaultRateProvider.sol L235-L247

```
235 function _applyAdjustment(uint256 amount, uint56 adjustmentRate, Adjustment
236     if (adjustmentRate == 0) {
237         return amount;
238     }
239     if (adjustment == Adjustment.DOWN) {
240         return Math.mulDiv(amount, SCALE - uint256(adjustmentRate), SCALE, M
241     }
242     if (adjustment == Adjustment.UP) {
243         // Inverse of DOWN: ceil(amount * SCALE / (SCALE - adjustmentRate))
244         return Math.mulDiv(amount, SCALE, SCALE - uint256(adjustmentRate), M
245     }
246     revert UnknownAdjustment();
247 }
```

RECOMMENDATION

We advise it to be omitted, and the `if` conditionals to be simplified.

• ACKNOWLEDGED d0f8fd98b7

The UltraYield team assessed this exhibit and opted to retain the unreachable code as a forward-compatibility guard.

CODE STYLE VPM-C

VaultPriceManager Code Style Findings

Contains all the findings that relate to code style on the contract codebase.

src/oracles/VaultPriceManager.sol

VPM-01C INFO

Inefficient Event Emission

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Alleviated	VaultPriceManager.sol:L151, L153

DESCRIPTION

The referenced event emission can be optimized by emitting it before the variable is mutated.

AFFECTED CODE 1 instance

src/oracles/VaultPriceManager.sol L148-L154

```
148  /// @notice Update the activation lead time (bounded).
149  function setDelay(uint256 newDelay) external onlyOwner {
150      require(newDelay ≥ MIN_DELAY && newDelay ⩽ MAX_DELAY, InvalidDelay());
151      uint256 oldDelay = delay;
152      delay = newDelay;
153      emit DelayUpdated(oldDelay, newDelay);
154  }
```

RECOMMENDATION

We advise this to be done so, utilizing the `delay` storage entry directly and avoiding the memory reservation of `oldDelay`.

• ALLEVIATED d0f8fd98b7

The inefficient event emission has been optimized as advised.

VPM-02C INFO

Inefficient `mapping` Lookups

TYPE	DISPOSITION	LOCATIONS
Gas Optimization	• Alleviated	I-1 VaultPriceManager.sol:L192, L195, L196 I-2 VaultPriceManager.sol:L202, L203

DESCRIPTION

The linked statements perform key-based lookup operations on `mapping` declarations from storage multiple times for the same key redundantly.

AFFECTED CODE 2 instances — excerpt shown

src/oracles/VaultPriceManager.sol L192-L196

```
192  require(destinations[selector].exists, DestinationNotFound());
193  require(receiver ≠ address(0), ZeroAddress());
194  require(gasLimit ≠ 0, InvalidGasLimit());
195  destinations[selector].receiver = receiver;
196  destinations[selector].gasLimit = gasLimit;
```

RECOMMENDATION

As the lookups internally perform an expensive `keccak256` operation, we advise the lookups to be cached wherever possible to a single local declaration that either holds the value of the `mapping` in case of primitive types or holds a `storage` pointer to the `struct` contained.

As the compiler's optimizations may take care of these caching operations automatically at-times, we advise the optimization to be selectively applied, tested, and then fully adopted to ensure that the proposed caching model indeed leads to a reduction in gas costs.

• ALLEVIATED d0f8fd98b7

All referenced inefficient `mapping` lookups have been optimized to the greatest extent possible, significantly reducing the gas cost of the functions the statements were located in.

VPM-03C INFO

Potentially Redundant Shift Operation

TYPE	DISPOSITION	LOCATION
Gas Optimization	• Alleviated	VaultPriceManager.sol:L180

DESCRIPTION

The `VaultPriceManager::removeDestination` will redundantly assign the last entry to itself if the `for` loop exhausts all entries.

AFFECTED CODE 1 instance

src/oracles/VaultPriceManager.sol L177-L184

```
177  uint256 len = destinationSelectors.length;
178  for (uint256 i; i < len; ++i) {
179      if (destinationSelectors[i] == selector) {
180          destinationSelectors[i] = destinationSelectors[len - 1];
181          destinationSelectors.pop();
182          break;
183      }
184  }
```

RECOMMENDATION

We advise the `for` loop to run until the `len - 1` value, and the `pop` instruction to be relocated after the `for` loop optimizing its cost.

• ALLEVIATED d0f8fd98b7

The `for` lookup iteration and associated pop instruction were optimized per our recommendation.

APPENDIX

Finding Types

A description of each finding type included in the report can be found below and is linked by each respective finding. A full list of finding types Omniscia has defined will be viewable at the central audit methodology we will publish soon.

External Call Validation	Many contracts that interact with DeFi contain a set of complex external call executions that need to happen in a particular sequence and whose execution is usually taken for granted whereby it is not always the case. External calls should always be validated, either in the form of `require` checks imposed at the contract-level or via more intricate mechanisms such as invoking an external getter-variable and ensuring that it has been properly updated.
Input Sanitization	As there are no inherent guarantees to the inputs a function accepts, a set of guards should always be in place to sanitize the values passed in to a particular function.
Indeterminate Code	These types of issues arise when a linked code segment may not behave as expected, either due to mistyped code, convoluted `if` blocks, overlapping functions / variable names and other ambiguous statements.

Language Specific	Language specific issues arise from certain peculiarities that the Solidity language boasts that discerns it from other conventional programming languages. For example, the EVM is a 256-bit machine meaning that operations on less-than-256-bit types are more costly for the EVM in terms of gas costs, meaning that loops utilizing a `uint8` variable because their limit will never exceed the 8-bit range actually cost more than redundantly using a `uint256` variable.
Code Style	An official Solidity style guide exists that is constantly under development and is adjusted on each new Solidity release, designating how the overall look and feel of a codebase should be. In these types of findings, we identify whether a project conforms to a particular naming convention and whether that convention is consistent within the codebase and legible. In case of inconsistencies, we point them out under this category. Additionally, variable shadowing falls under this category as well which is identified when a local-level variable contains the same name as a contract-level variable that is present in the inheritance chain of the local execution level's context.
Gas Optimization	Gas optimization findings relate to ways the codebase can be optimized to reduce the gas cost involved with interacting with it to various degrees. These types of findings are completely optional and are pointed out for the benefit of the project's developers.
Standard Conformity	These types of findings relate to incompatibility between a particular standard's implementation and the project's implementation, oftentimes causing significant issues in the usability of the contracts.

Mathematical Operations	In Solidity, math generally behaves differently than other programming languages due to the constraints of the EVM. A prime example of this difference is the truncation of values during a division which in turn leads to loss of precision and can cause systems to behave incorrectly when dealing with percentages and proportion calculations.
Logical Fault	This category is a bit broad and is meant to cover implementations that contain flaws in the way they are implemented, either due to unimplemented functionality, unaccounted-for edge cases or similar extraordinary scenarios.
Centralization Concern	This category covers all findings that relate to a significant degree of centralization present in the project and as such the potential of a Single-Point-of-Failure (SPoF) for the project that we urge them to re-consider and potentially omit.
Reentrant Call	This category relates to findings that arise from re-entrant external calls (such as EIP-721 minting operations) and revolve around the inapplicacy of the Checks-Effects-Interactions (CEI) pattern, a pattern that dictates checks (`require` statements etc.) should occur before effects (local storage updates) and interactions (external calls) should be performed last.

APPENDIX

Severity Definition

In the ever-evolving world of blockchain technology, vulnerabilities continue to take on new forms and arise as more innovative projects manifest, new blockchain-level features are introduced, and novel layer-2 solutions are launched. When performing security reviews, we are tasked with classifying the various types of vulnerabilities we identify into subcategories to better aid our readers in understanding their impact.

Within this page, we will clarify what each severity level stands for and our approach in categorizing the findings we pinpoint in our audits.

> ! To note, all severity assessments are performed **as if the contract's logic cannot be upgraded** regardless of the underlying implementation.

Severity Matrix

There are five distinct severity levels within our reports; `unknown`, `informational`, `minor`, `medium`, and `major`. A TL;DR overview table can be found below as well as a dedicated chapter to each severity level:

	IMPACT NONE	IMPACT LOW	IMPACT MODERATE	IMPACT HIGH
LIKELIHOOD NONE	Informational	Informational	Informational	Informational
LIKELIHOOD LOW	Informational	Minor	Minor	Medium
LIKELIHOOD MODERATE	Informational	Minor	Medium	Major
LIKELIHOOD HIGH	Informational	Medium	Major	Major

Severity Levels

Unknown Severity

The `unknown` severity level is reserved for misbehaviors we observe in the codebase that cannot be quantified using the above metrics. Examples of such vulnerabilities include potentially desirable system behavior that is undocumented, reliance on external dependencies that are out-of-scope but could result in some form of vulnerability arising, use of external out-of-scope contracts that appears incorrect but cannot be pinpointed, and other such vulnerabilities.

In general, `unknown` severity level vulnerabilities require follow-up information by the project being audited and are either adjusted in severity (if valid), or marked as nullified (if invalid).

Additionally, the `unknown` severity level is sometimes assigned to centralization issues that cannot be assessed in likelihood due to their exploitation being tied to the honesty of the project's team.

Informational Severity

The `informational` severity level is dedicated to findings that do not affect the code functionally and tend to be stylistic or optimizational in nature. Certain edge cases are also set under `informational` vulnerabilities, such as overflow operations that will not manifest in the lifetime of the contract but should be guarded against as a best practice, to give an example.

Minor Severity

The `minor` severity level is meant for vulnerabilities that require functional changes in the code but tend to either have little impact or be unlikely to be recreated in a production environment. These findings can be acknowledged except for findings with a moderate impact but low likelihood which must be alleviated.

Medium Severity

The `medium` severity level is assigned to vulnerabilities that must be alleviated and have an observable impact on the overall project. These findings can only be acknowdged if the project deems them desirable behavior and we disagree with their point-of-view, instead urging them to reconsider their stance while marking the exhibit as acknowledged given that the project has ultimate say as to what vulnerabilities they end up patching in their system.

Major Severity

The `major` severity level is the maximum that can be specified for a finding and indicates a significant flaw in the code that must be alleviated.

Likelihood & Impact Assessment

As the preface chapter specifies, the blockchain space is constantly reinventing itself meaning that new vulnerabilities take place and our understanding of what security means differs year-to-year.

In order to reliably assess the likelihood and impact of a particular vulnerability, we instead apply an abstract measurement of a vulnerability's impact, duration the impact is applied for, and probability that the vulnerability would be exploited in a production environment.

Our proposed definitions are inspired by multiple sources in the security community and are as follows:

IMPACT

HIGH

A core invariant of the protocol can be broken for an extended duration.

MODERATE

A non-core invariant of the protocol can be broken for an extended duration or at scale, or an otherwise major-severity issue is reduced due to hypotheticals or external factors affecting likelihood.

LIKELIHOOD

HIGH

A flaw in the code that can be exploited trivially and is ever-present.

MODERATE

A flaw in the code that requires some external factors to be exploited that are likely to manifest in practice.

LOW

A non-core invariant of the protocol can be broken with reduced likelihood or impact.

NONE

A code or documentation flaw whose impact does not achieve low severity, or an issue without theoretical impact; a valuable best-practice

LOW

A flaw in the code that requires multiple external factors to be exploited that may manifest in practice but would be unlikely to do so.

NONE

A flaw in the code that requires external factors proven to be impossible in a production environment, either due to mathematical constraints, operational constraints, or system-related factors (i.e. EIP-20 tokens not being re-entrant).

APPENDIX

Disclaimer

The following disclaimer applies to all versions of the audit report produced (preliminary / public / private) and is in effect for all past, current, and future audit reports that are produced and hosted under Omniscia:

IMPORTANT TERMS & CONDITIONS REGARDING OUR SECURITY AUDITS/REVIEWS/REPORTS AND ALL PUBLIC/PRIVATE CONTENT/DELIVERABLES

Omniscia ("Omniscia") has conducted an independent security review to verify the integrity of and highlight any vulnerabilities, bugs or errors, intentional or unintentional, that may be present in the codebase that were provided for the scope of this Engagement.

Blockchain technology and the cryptographic assets it supports are nascent technologies. This makes them extremely volatile assets. Any assessment report obtained on such volatile and nascent assets may include unpredictable results which may lead to positive or negative outcomes.

In some cases, services provided may be reliant on a variety of third parties. This security review does not constitute endorsement, agreement or acceptance for the Project and technology that was reviewed. Users relying on this security review should not consider this as having any merit for financial advice or technological due diligence in any shape, form or nature.

The veracity and accuracy of the findings presented in this report relate solely to the proficiency, competence, aptitude and discretion of our auditors. Omniscia and its employees make no guarantees, nor assurance that the contracts are free of exploits, bugs, vulnerabilities, deprecation of technologies or any system / economical / mathematical malfunction.

This audit report shall not be printed, saved, disclosed nor transmitted to any persons or parties on any objective, goal or justification without due written assent, acquiescence or approval by Omniscia.

All the information/opinions/suggestions provided in this report does not constitute financial or investment advice, nor should it be used to signal that any person reading this report should invest their funds without sufficient individual due diligence regardless of the findings presented in this report.

Information in this report is provided 'as is'. Omniscia is under no covenant to the completeness, accuracy or solidity of the contracts reviewed. Omniscia's goal is to help reduce the attack vectors/surface and the high level of variance associated with utilizing new and consistently changing technologies.

Omniscia in no way claims any guarantee, warranty or assurance of security or functionality of the technology that was in scope for this security review.

In no event will Omniscia, its partners, employees, agents or any parties related to the design/creation of this security review be ever liable to any parties for, or lack thereof, decisions and/or actions with regards to the information provided in this security review.

Cryptocurrencies and all other technologies directly or indirectly related to cryptocurrencies are not standardized, highly prone to malfunction and extremely speculative by nature. No due diligence and/or safeguards may be insufficient and users should exercise maximum caution when participating and/or investing in this nascent industry.

The preparation of this security review has made all reasonable attempts to provide clear and actionable recommendations to the Project team (the “client”) with respect to the rectification, amendment and/or revision of any highlighted issues, vulnerabilities or exploits within the contracts in scope for this engagement.

It is the sole responsibility of the Project team to provide adequate levels of test and perform the necessary checks to ensure that the contracts are functioning as intended, and more specifically to ensure that the functions contained within the contracts in scope have the desired intended effects, functionalities and outcomes, as documented by the Project team.

All services, the security reports, discussions, work product, attack vectors description or any other materials, products or results of this security review engagement is provided "as is" and "as available" and with all faults, uncertainty and defects without warranty or guarantee of any kind.

Omniscia will assume no liability or responsibility for delays, errors, mistakes, or any inaccuracies of content, suggestions, materials or for any loss, delay, damage of any kind which arose as a result of this engagement/security review.

Omniscia will assume no liability or responsibility for any personal injury, property damage, of any kind whatsoever that resulted in this engagement and the customer having access

to or use of the products, engineers, services, security report, or any other other materials.

For avoidance of doubt, this report, its content, access, and/or usage thereof, including any associated services or materials, shall not be considered or relied upon as any form of financial, investment, tax, legal, regulatory, or any other type of advice.

© 2026 Omniscia · revision e0022c003c · ❖ c02b898461